

Retrieval-Augmented Generation (RAG)

Understanding the concept, its purpose, and its types in Large Language Models

1. What is RAG?

Retrieval-Augmented Generation, commonly known as RAG, is a technique that combines a Large Language Model (LLM) with an external knowledge retrieval system. Instead of relying purely on the knowledge an LLM learned during its training, RAG allows the model to fetch relevant information from an external source — such as a document database, a vector store, or the internet — at the time a question is asked, and then use that retrieved information to generate a more accurate and grounded answer.

In simple terms, RAG works in two steps: first, it retrieves relevant pieces of information related to the user's query from an external knowledge base; second, it augments the prompt given to the LLM with this retrieved information so that the model can generate a response that is factually grounded and up to date, rather than depending solely on what it memorized during training.

2. Why is RAG Used in LLMs?

LLMs are trained on large datasets up to a certain cutoff date. Once trained, their internal knowledge becomes fixed — they cannot automatically learn new facts, updates, or events that happen afterward. This creates several limitations that RAG is designed to solve:

- Outdated knowledge: LLMs cannot know about events, data, or facts that occurred after their training cutoff.
- Hallucination: When an LLM does not know an answer, it may generate plausible-sounding but incorrect information.
- Lack of domain-specific or private knowledge: LLMs are trained on general public data and have no knowledge of a company's internal documents, private databases, or specialized records.
- High cost of retraining: Updating an LLM's internal knowledge through retraining or fine-tuning every time new information appears is expensive and time-consuming.

RAG addresses these problems by letting the model pull fresh, relevant, and verifiable information from an external source at the moment a query is made, rather than depending only on what was baked into the model's parameters during training.

3. Why RAG is Used Instead of Fine-Tuning

Fine-tuning is the process of taking a pre-trained LLM and training it further on a specific dataset so that it adapts its internal parameters (weights) to new patterns, styles, or knowledge. While fine-tuning is useful for teaching a model a new skill, tone, or behavior, it is not efficient for keeping a model

updated with constantly changing information.

Key difference: Static vs Dynamic Knowledge

Fine-tuning is essentially a static process. Once a model is fine-tuned on a dataset, that knowledge becomes frozen inside its parameters. If new information appears the next day, the model still does not know about it unless it is fine-tuned again — which requires retraining, computing resources, and time.

RAG, on the other hand, is a dynamic process. Since the knowledge source used by RAG is external (such as a document store or database), it can be updated at any time — by simply adding, editing, or removing documents in the knowledge base. The LLM does not need to be retrained at all; the very next query will automatically retrieve the updated information from the knowledge base.

- Fine-tuning: knowledge is embedded inside model weights; updating it requires retraining the model.
- RAG: knowledge lives outside the model in a retrievable store; updating it only requires updating that store.
- Fine-tuning is costly and slow to repeat for every new piece of information.
- RAG is fast, flexible, and cost-efficient since no retraining is needed for new or changing data.
- Fine-tuning is better suited for teaching style, tone, format, or specialized reasoning patterns.
- RAG is better suited for keeping factual, frequently changing, or domain-specific knowledge up to date.

This is why, for use cases where information changes frequently — such as company policies, news, product catalogs, or technical documentation — RAG is generally preferred over fine-tuning. Fine-tuning remains the better choice when the goal is to change how a model behaves or communicates, not what facts it knows.

4. Types of RAG

RAG is not a single fixed technique; it has evolved into several types depending on how retrieval and generation are combined. The major types are explained below.

4.1 Naive RAG (Basic RAG)

This is the simplest and most common form of RAG. The user's query is converted into a vector embedding, relevant documents are retrieved from a vector database using similarity search, and these documents are directly added to the prompt sent to the LLM. The LLM then generates an answer based on the retrieved context.

Why it is used: It is simple to implement and works well for straightforward question-answering tasks where one round of retrieval is enough to find the relevant information.

4.2 Advanced RAG

Advanced RAG improves upon naive RAG by adding extra steps before and after retrieval. Before retrieval, the query may be rewritten, expanded, or broken into sub-queries to improve search accuracy. After retrieval, a re-ranking step is used to filter out irrelevant or low-quality documents, keeping only the most useful ones before passing them to the LLM.

Why it is used: It increases the accuracy and relevance of the retrieved information, reducing noise and improving the quality of the final answer, especially for complex or ambiguous queries.

4.3 Modular RAG

Modular RAG breaks the RAG pipeline into independent, swappable components — such as separate modules for query routing, retrieval, re-ranking, summarization, and generation. Each module can be modified, replaced, or improved without affecting the rest of the pipeline.

Why it is used: It offers flexibility and scalability, allowing developers to customize the pipeline for specific tasks, mix different retrieval strategies, and easily upgrade individual parts of the system over time.

4.4 Graph RAG

Graph RAG uses a knowledge graph instead of, or alongside, a vector database. Information is stored as entities and relationships (nodes and edges), and retrieval involves traversing these relationships to gather connected, structured context rather than isolated chunks of text.

Why it is used: It is helpful for queries that require understanding relationships between multiple entities or multi-hop reasoning, where simple similarity search on text chunks is not enough.

4.5 Hybrid RAG

Hybrid RAG combines multiple retrieval techniques together — for example, combining keyword-based search (such as BM25) with vector-based semantic search, or combining structured database queries with unstructured document retrieval.

Why it is used: Different retrieval methods have different strengths; combining them improves overall retrieval accuracy and ensures that both exact keyword matches and semantically similar content are captured.

4.6 Agentic RAG

Agentic RAG introduces an autonomous decision-making layer (an agent) that decides when to retrieve, what source to retrieve from, whether multiple retrieval steps are needed, and how to combine information from different tools or knowledge bases before generating a final response.

Why it is used: It is suited for complex, multi-step tasks where a single retrieval step is not sufficient, and the system needs to reason about which sources to query and in what order.

5. Summary

RAG is used in LLMs to overcome the limitations of static, fixed training knowledge by allowing models to access fresh, accurate, and domain-specific information at query time. Compared to fine-tuning, which embeds knowledge permanently into model weights and requires retraining for every update, RAG keeps knowledge external and easily updatable, making it a dynamic and cost-efficient solution. As RAG has matured, it has evolved into multiple types — Naive, Advanced, Modular, Graph, Hybrid, and Agentic RAG — each designed to handle different levels of complexity in retrieval and reasoning, giving developers the flexibility to choose the right approach based on the nature of their data and the complexity of the queries they need to answer.

6. How a Typical RAG Pipeline Works (Step by Step)

To understand RAG more concretely, it helps to walk through the typical sequence of steps a RAG system follows when answering a user's question.

- Document ingestion: Source documents (PDFs, web pages, database records, etc.) are collected and split into smaller chunks of text.
- Embedding: Each chunk is converted into a numerical vector representation using an embedding model, capturing its semantic meaning.
- Indexing: These vectors are stored in a vector database (or similar index) so they can be searched efficiently later.
- Query embedding: When a user asks a question, the query itself is also converted into a vector using the same embedding model.
- Retrieval: The system searches the index for chunks whose vectors are most similar to the query vector, returning the most relevant pieces of text.
- Augmentation: The retrieved chunks are inserted into the prompt along with the original user question, giving the LLM extra context.
- Generation: The LLM reads the augmented prompt and generates a final answer that is grounded in the retrieved information.

This pipeline allows the LLM to behave as if it always has access to the latest and most relevant information, even though the model's own internal weights are never changed.

7. Key Benefits of RAG

- Up-to-date answers: Knowledge can be refreshed simply by updating the external data source, without touching the model.
- Reduced hallucination: Because answers are grounded in retrieved, real documents, the model is less likely to invent facts.
- Source traceability: RAG systems can show which documents were used to generate an answer, improving transparency and trust.
- Domain adaptability: The same base LLM can be applied to many different domains just by changing the knowledge base it retrieves from.
- Lower cost: Avoids the repeated computational expense of retraining or fine-tuning a model every time information changes.

8. When to Combine RAG with Fine-Tuning

In practice, RAG and fine-tuning are not always mutually exclusive — they solve different problems and can be combined. Fine-tuning can be used to teach the model a consistent tone, a specific output format, or specialized reasoning ability, while RAG is used on top of that fine-tuned model to supply current, factual, or domain-specific knowledge at query time. This combination allows a system to have both the right behavior (from fine-tuning) and the right knowledge (from RAG).